

GHIBLI TASKS

README_FOR_CLAUDE

Version 1.0

Objective

Create a complete Full Stack web application for task management inspired by the visual style of Studio Ghibli.

The project must prioritize:

- clean architecture
- readable code
- scalability
- reusable components
- responsive design
- accessibility
- performance
- maintainability

The application should be developed as if it were a real product ready for production and portfolio presentation.

Project Philosophy

This application should prioritize user experience over feature quantity.

Whenever a design or technical decision is required, prefer:

simplicity

clarity

maintainability

consistency

Avoid unnecessary complexity. Every feature should feel intentional.

Less is better than more.

Technology Stack

The project must use:

Frontend

- Next.js (App Router)
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- Lucide React

Backend

- Next.js API Routes
- Server Actions whenever appropriate

Database

- PostgreSQL
- Prisma ORM

Authentication

- Better Auth

Deployment

- GitHub
- Vercel
- Neon PostgreSQL

Package Manager

- npm

Lint

- ESLint

Formatting

- Prettier

Architecture

Use Feature Based Architecture.

Example:

src/

app/

components/

features/

hooks/

lib/

services/

types/

utils/

styles/

prisma/

public/

Avoid placing everything inside "components".

Organize components according to their responsibility.

Coding Standards

Always follow these principles.

- SOLID
- Clean Code
- DRY
- KISS

Avoid duplicated code.

Always prefer reusable components.

Never create giant files.

Keep components small.

Naming Convention

Variables

camelCase

Functions

camelCase

Components

PascalCase

Interfaces

PascalCase

Enums

PascalCase

Database tables

snake_case

Database columns

snake_case

Files

Prefer kebab-case

Component Guidelines

Components should:

- have a single responsibility
 - be reusable
 - receive only necessary props
 - avoid deeply nested JSX
 - avoid unnecessary state
 - extract business logic into hooks or services
 - prefer composition over inheritance
-

Page Guidelines

Pages should:

remain lightweight

delegate UI to reusable components

delegate logic to services

avoid direct database access

avoid duplicated layouts

focus only on composition

Folder Rules

Never place business logic inside UI components.

Business logic should stay inside:

services

or

lib

Keep components responsible only for rendering.

Component Rules

Components should:

- receive props
 - avoid duplicated layouts
 - avoid unnecessary state
 - be reusable
-

TypeScript

Never use:

any

Prefer:

- interfaces
- types
- generics

Use strict typing.

Styling

Use only Tailwind CSS.

Avoid inline styles.

Avoid CSS Modules unless absolutely necessary.

Create reusable utility classes when appropriate.

Responsive Design

The website must work well on:

Desktop

Tablet

Mobile

Mobile-first approach.

Accessibility

Always include:

aria-label

semantic HTML

keyboard navigation

focus states

color contrast

Performance

Prefer:

Server Components

Lazy Loading

Image Optimization

Code Splitting

Avoid unnecessary renders.

Database

Use Prisma ORM.

Use migrations.

Never modify the database manually if Prisma migration can do it.

API

Organize API routes clearly.

Return standardized responses.

Use:

success

message

data

error

Handle exceptions.

Never expose internal errors to users.

Authentication

Authentication should support:

Register

Login

Logout

Protected routes

Session persistence

Password hashing

Validation

Use Zod.

Validate:

Forms

API

Database input

Never trust client-side data.

Error Handling

Always:

Handle errors gracefully.

Display friendly messages.

Log technical errors separately.

Git

Use meaningful commits.

Example:

feat:

fix:

refactor:

style:

docs:

test:

chore:

Documentation

Document:

complex logic

utilities

hooks

API endpoints

Do not comment obvious code.

UI Theme

The application must be heavily inspired by Studio Ghibli.

The goal is to create a calm, cozy and magical experience.

The interface should feel:

- soft
- welcoming
- peaceful
- minimal
- nostalgic

Avoid a childish appearance.

Aim for elegant simplicity.

Visual Identity

Primary colors

Soft Green

Warm Cream

Forest Green

Warm Brown

Light Beige

Muted Olive

Avoid saturated colors.

Design Tokens

Token	Usage
Primary	Forest Green
Secondary	Soft Green
Background	Warm Cream
Surface	Light Beige
Accent	Muted Olive
Success	Moss Green
Warning	Warm Amber
Danger	Soft Red
Text Primary	Dark Olive
Text Secondary	Warm Brown

Visual References

Use the provided reference images as inspiration.

Important:

Do not copy artwork directly.

Only reproduce:

- atmosphere
- color palette
- spacing

- rounded shapes
- visual rhythm
- cozy feeling

Icons

Prefer:

Lucide

Simple icons

Rounded appearance

Animations

Animations should be subtle.

Examples:

Fade

Slide

Scale

Hover

Duration between:

150ms

300ms

Avoid exaggerated animations.

Borders

Prefer rounded corners.

Buttons

Cards

Inputs

Dialogs

Should all have a soft appearance.

Shadows

Soft shadows.

Never use harsh shadows.

General UX

The interface should always feel:

Simple

Calm

Clean

Comfortable

Never overwhelming.

Design Principles

White space is important.

Do not overcrowd the interface.

Prioritize readability.

Maintain consistent spacing.

Decision Making Rules ← (novo)

Whenever multiple implementation options exist:

Prefer simplicity.

Prefer maintainability.

Prefer readability.

Prefer native Next.js features.

Avoid unnecessary dependencies.

Avoid overengineering.

AI Development Rules

Whenever creating a new feature:

1. Think about architecture first.
2. Reuse existing components whenever possible.
3. Avoid duplicate logic.

4. Maintain consistent folder organization.
5. Respect the project's design language.
6. Keep code clean and modular.
7. If an implementation choice is uncertain, prefer maintainability over cleverness.

Never rewrite unrelated files unless necessary.

Before creating a new component, verify whether an existing one can be reused.

Always preserve project consistency.

Development Roadmap

General Rules

Each phase represents a complete milestone.

Before moving to the next phase, ensure:

- The application builds without errors.
 - No TypeScript errors exist.
 - No ESLint errors exist.
 - Existing features continue working.
 - The UI remains consistent with the project's design system.
 - Components are reused whenever possible.
 - The code is committed only after the phase is complete.
-

Phase 1 — Project Foundation

Goal

Create the base project structure and development environment.

Tasks

- Initialize the Next.js project.
- Configure TypeScript.
- Configure TailwindCSS.
- Install shadcn/ui.
- Configure ESLint.
- Configure Prettier.
- Configure aliases.

- Configure environment variables.
- Create the folder structure.
- Configure Prisma.
- Connect PostgreSQL.
- Create the first migration.
- Configure GitHub repository.
- Prepare Vercel deployment compatibility.

Deliverables

- Running application.
- Database connected.
- Clean folder architecture.
- Initial commit.

Phase 2 — Design System

Goal

Create the visual foundation of the application.

Tasks

Create reusable UI components.

Components should include:

- Button
- Input
- Textarea
- Card
- Badge
- Modal
- Dropdown
- Avatar
- Tooltip
- Toast
- Dialog
- Skeleton

- Loading Spinner

Create:

- Typography system
- Color palette
- Shadows
- Border radius
- Spacing system
- Animation utilities

Deliverables

A reusable design system.

No page-specific components yet.

Phase 3 — Layout

Goal

Create the application shell.

Tasks

Develop:

- Navbar
- Sidebar
- Mobile Navigation
- Main Layout
- Footer
- User Menu

Implement:

- Responsive layout
- Navigation routing
- Theme consistency

Deliverables

The application layout should be complete.

No business logic yet.

Phase 4 — Authentication

Goal

Allow users to securely access the application.

Features

- Register
- Login
- Logout
- Protected routes
- Session persistence
- Password hashing
- User profile

Database

Create User model.

Fields:

- id
- name
- email
- password
- createdAt
- updatedAt

Deliverables

A complete authentication flow.

Phase 5 — Database Models

Goal

Create all database entities before implementing features.

Models:

User

Task

Category

Tag

Subtask

Reminder

Attachment

Deliverables

Complete Prisma schema.

All migrations created.

Phase 6 — Task Management

Goal

Implement the core functionality.

Features

Create task

Edit task

Delete task

Complete task

Restore task

Archive task

Duplicate task

Pin task

Search tasks

Filter tasks

Sort tasks

Task fields

Title

Description

Priority

Category

Tags

Due date

Reminder

Status

Created At

Updated At

Completed At

Phase 7 — Categories

Features

Create category

Edit category

Delete category

Assign color

Assign icon

Filter by category

Phase 8 — Tags

Features

Create tags

Delete tags

Assign multiple tags

Filter by tags

Search by tags

Phase 9 — Subtasks

Features

Create subtasks

Edit subtasks

Delete subtasks

Progress calculation

Completion percentage

Collapse / Expand list

Phase 10 — Calendar

Features

Monthly view

Weekly view

Daily view

Upcoming tasks

Overdue tasks

Today tasks

Due date editing

Phase 11 — Dashboard

Widgets

Today's Tasks

Completed Tasks

Pending Tasks

Weekly Progress

Monthly Progress

Upcoming Deadlines

Most Productive Day

Recent Activity

Phase 12 — Notifications

Features

Reminder system

Toast notifications

Due soon alerts

Overdue alerts

Success messages

Error messages

Phase 13 — Search

Features

Global search

Search suggestions

Filters

Recent searches

Search by:

Title

Description

Category

Tag

Status

Priority

Phase 14 — Settings

User Settings

Profile

Avatar

Password

Theme

Language

Notification preferences

Phase 15 — Polish

Improve UX

Loading skeletons

Empty states

Animations

Transitions

Error pages

404 page

500 page

Success states

Accessibility improvements

Performance improvements

Phase 16 — Deployment

Final Checklist

- Environment variables configured.
 - Production database connected.
 - Vercel configured.
 - Build passes.
 - SEO metadata configured.
 - Open Graph configured.
 - Favicon configured.
 - Robots.txt
 - Sitemap
 - README completed.
-

Future Features

These features should NOT be implemented now.

Keep the architecture prepared for them.

Possible future additions:

- Recurring tasks
- Pomodoro timer
- Habit tracker
- Notes
- Markdown editor
- File attachments
- Google Calendar sync
- Discord notifications
- Email reminders
- AI task suggestions
- Dark mode

- Offline mode (PWA)
 - Multi-language support
 - Team collaboration
 - Shared task lists
-

Development Workflow

For every new feature, follow this exact order:

1. Analyze the requested feature.
2. Verify whether reusable components already exist.
3. If needed, create new reusable components first.
4. Update the database schema if necessary.
5. Update Prisma migrations.
6. Create backend logic.
7. Create API endpoints or Server Actions.
8. Validate data using Zod.
9. Implement the frontend UI.
10. Connect frontend to backend.
11. Handle loading, empty and error states.
12. Make the layout responsive.
13. Test the feature manually.
14. Refactor duplicated code.
15. Update documentation if necessary.

Never skip any step.

Portfolio Quality Requirement

This application is intended to be a professional portfolio project.

Every implementation should prioritize:

- readability
- scalability
- maintainability
- modular architecture

- reusable components
- clean UI
- accessibility
- production-ready quality

Avoid quick fixes or temporary solutions.

Whenever there are multiple implementation options, choose the one that best reflects modern Full Stack development best practices.

Development Behavior

Act as the lead software engineer of this project.

Before implementing any feature:

analyze the current architecture

identify reusable code

think about scalability

keep the codebase consistent

If an implementation may affect other parts of the application, explain the impact before making changes.

Avoid unnecessary refactors.

Never rewrite working code simply to use a different approach.

Preserve backward compatibility whenever possible.